Appendix B — Longitudinal Analysis of Rating Growth (Lichess)

Tina Huynh

2025-10-16

Contents

0.1 Appendix B — Longitudinal Analysis of Rating Growth (Lichess) 1

1 Summary statistics for growth rates **11**

0.1 Appendix B — Longitudinal Analysis of Rating Growth (Lichess)

0.1.1 B.1 Data Acquisition

```
library(tidyverse)
library(jsonlite)
library(lubridate)
library(httr)
library(dplyr)

# First, try to load existing rating history data
rating_hist_path <- "../../data/lichess_rating_history.csv"
existing_data <- NULL

if (file.exists(rating_hist_path)) {
    existing_data <- read_csv(rating_hist_path, show_col_types = FALSE)
    if (nrow(existing_data) > 0) {
        cat(sprintf(
            "Loaded existing rating history: %d records for %d users\n",
            nrow(existing_data), n_distinct(existing_data$user)
        ))
    }
}
```

```
## Loaded existing rating history: 1173 records for 5 users
```

```
# Only fetch new data if we don't have existing data
if (is.null(existing_data) || nrow(existing_data) == 0) {
    cat("No existing data found. Fetching from Lichess API...\n")

    # Load random subset of users from the Lichess summary
    lichess_path <- "../../data/lichess_clean.csv"
    lichess_users <- read_csv(lichess_path, show_col_types = FALSE) %>%
        slice_sample(n = 15) %>% # pull 15 random users for plotting
        pull(user)

    # Debug: Check selected users
```

```
cat("Selected users:\n")
print(lichess_users)

# Function to retrieve rating history for a given user
get_rating_history <- function(username) {
    if (is.na(username) || nchar(trimws(username)) == 0) {
        cat("  Skipping invalid username\n")
        return(NULL)
    }
    cat(paste0("\nFetching data for user: ", username, "\n"))
    Sys.sleep(runif(1, 0.5, 1.2)) # polite delay

    url <- paste0("https://lichess.org/api/user/", username, "/rating-history")

    tryCatch(
        {
            # Use httr for better error handling and timeout control
            response_raw <- GET(url, timeout(30))

            if (status_code(response_raw) != 200) {
                cat(paste0("  HTTP Error: ", status_code(response_raw), "\n"))
                return(NULL)
            }

            response <- fromJSON(content(response_raw, "text"), simplifyDataFrame = FALSE)
            cat(paste0("  Response length: ", length(response), "\n"))

            if (length(response) == 0) {
                cat("  No rating history available\n")
                return(NULL)
            }
            # Pre-allocate list with known size
            result_list <- vector("list", length(response))
            valid_entries <- 0

            # Vectorized processing where possible
            for (i in seq_along(response)) {
                item <- response[[i]]
                game_type <- item[["name"]]
                points_data <- item[["points"]]

                # Early validation
                if (is.null(points_data) || length(points_data) == 0 || is.null(game_type)) {
                    next
                }

                # More efficient matrix conversion
                tryCatch(
                    {
                        if (is.list(points_data) && all(lengths(points_data) == 2)) {
                            # Direct matrix creation from list
                            points_matrix <- do.call(rbind, points_data)
                        } else if (is.matrix(points_data) && ncol(points_data) == 2) {
```

```
                            points_matrix <- points_data
                        } else {
                            next
                        }

                        # Validate matrix dimensions
                        if (nrow(points_matrix) == 0) next

                        valid_entries <- valid_entries + 1

                        # More efficient tibble creation
                        result_list[[valid_entries]] <- tibble(
                            user = username,
                            category = game_type,
                            date_index = as.integer(points_matrix[, 1]),
                            rating = as.integer(points_matrix[, 2])
                        ) %>%
                            mutate(date = as_date("2013-01-01") + weeks(date_index))
                    },
                    error = function(e) {
                        cat(paste0("  Warning: Skipping malformed data for ", game_type, ": ", e$mes
                    }
                )
            }

            # Only bind valid entries
            if (valid_entries > 0) {
                result_list <- result_list[1:valid_entries]
                return(bind_rows(result_list))
            } else {
                cat("  No valid rating points found\n")
                return(NULL)
            }
        },
        error = function(e) {
            cat(paste0("  Error: ", e$message, "\n"))
            return(NULL)
        }
    )
}

# Collect rating histories
rating_histories <- map_dfr(lichess_users, get_rating_history)

# Check if we got any data
if (is.null(rating_histories) || nrow(rating_histories) == 0) {
    cat("\nNo rating history data collected. Check API connectivity or try different users.\n")
    # Create empty data frame with correct structure
    rating_histories <- tibble(
        user = character(),
        category = character(),
        date_index = integer(),
        rating = integer(),
```

```
            date = as.Date(character())
        )
    } else {
        cat(sprintf(
            "\nSuccessfully collected %d rating records for %d users\n",
            nrow(rating_histories), n_distinct(rating_histories$user)
        ))
        # Save the newly collected data
        write_csv(rating_histories, "../../data/lichess_rating_history.csv")
    }
} else {
    # Use existing data
    rating_histories <- existing_data
}

glimpse(rating_histories)
```

```
## Rows: 1,173
## Columns: 5
## $ user       <chr> "Magnus_Carlsen", "Magnus_Carlsen", "Magnus_Carlsen", "Magn~
## $ category   <chr> "Blitz", "Blitz", "Blitz", "Blitz", "Blitz", "Blitz", "Blit~
## $ date_index <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
## $ rating     <dbl> 1696, 1677, 1675, 1675, 1686, 1684, 1646, 1611, 1633, 1630,~
## $ date       <date> 2013-01-08, 2013-01-15, 2013-01-22, 2013-01-29, 2013-02-05~
```

0.1.2 B.2 Time-Series Cleaning and Aggregation

```
library(zoo)
```

```
##
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```
# Check if we have data to process
if (nrow(rating_histories) == 0) {
    cat("No data available for time-series analysis. Skipping this section.\n")
    rating_summary <- tibble()
    category_stats <- tibble()
} else {
    # Clean, filter, and aggregate by category (e.g., Blitz, Rapid)
    rating_summary <- rating_histories %>%
        filter(!is.na(rating), !is.na(user), !is.na(category)) %>%
        arrange(user, category, date_index) %>%
        group_by(user, category) %>%
        filter(n() >= 3) %>% # Ensure minimum data for rolling average
        mutate(
            relative_time = row_number(),
            rating_change = rating - lag(rating, default = first(rating)),
            cumulative_change = rating - first(rating),
            .before = everything()
```

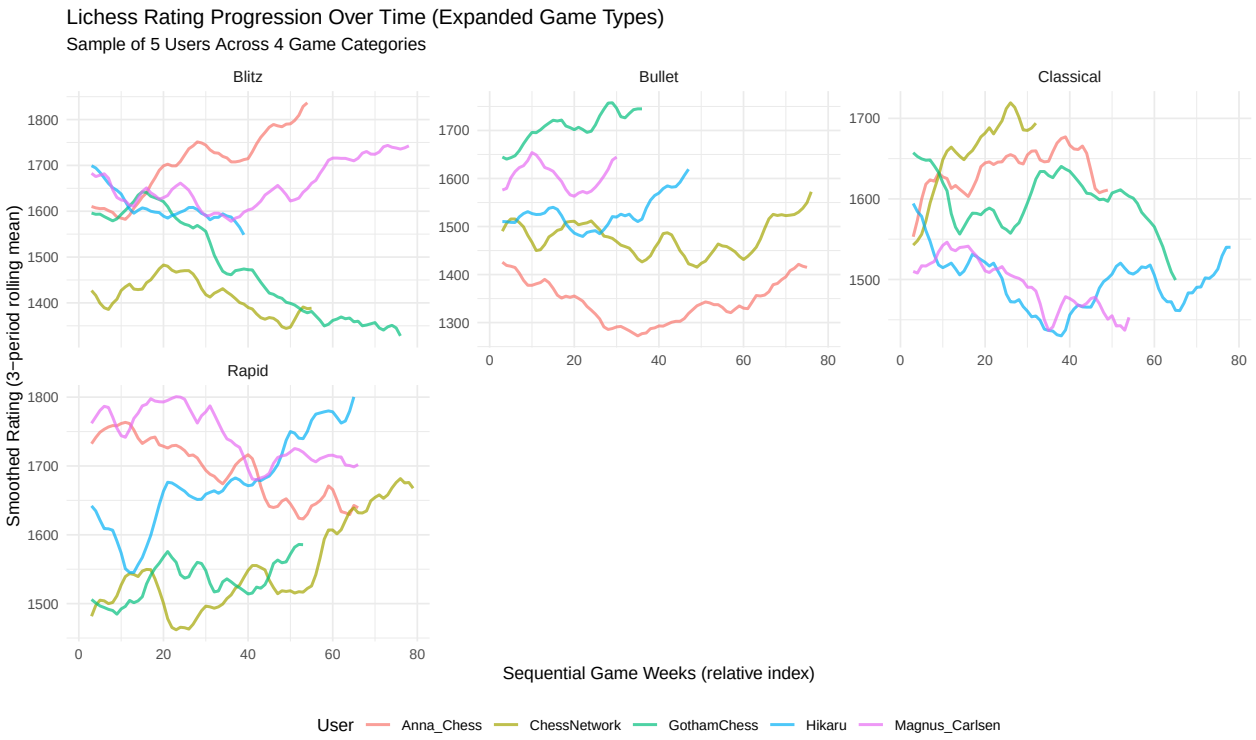
```
        ) %>%
        arrange(relative_time) %>%
        mutate(
            rating_smooth = case_when(
                n() >= 3 ~ zoo::rollapply(rating, width = 3, FUN = mean, fill = NA, align = "right"),
                TRUE ~ rating # Use original rating if insufficient data for smoothing
            )
        ) %>%
        ungroup()

    # Optimized summary statistics with single pass through data
    category_stats <- rating_summary %>%
        group_by(category) %>%
        summarise(
            users = n_distinct(user),
            observations = n(),
            mean_rating = mean(rating, na.rm = TRUE),
            sd_rating = sd(rating, na.rm = TRUE),
            mean_change = mean(rating_change[rating_change != 0], na.rm = TRUE), # Exclude initial zero
            median_rating = median(rating, na.rm = TRUE),
            .groups = "drop"
        ) %>%
        arrange(desc(users), desc(observations))

    # Enhanced table output with better formatting
    knitr::kable(
        category_stats,
        digits = 2,
        caption = "Rating Summary Statistics by Game Category",
        col.names = c("Category", "Users", "Obs", "Mean Rating", "SD", "Avg Change/Week", "Median Rating
        format.args = list(big.mark = ",")
    )

    head(rating_summary, 10)
}
```

```
## # A tibble: 10 x 9
##    relative_time rating_change cumulative_change user        category date_index
##            <int>         <dbl>             <dbl> <chr>       <chr>         <dbl>
##  1             1             0                 0 Anna_Chess  Blitz             1
##  2             1             0                 0 Anna_Chess  Bullet            1
##  3             1             0                 0 Anna_Chess  Classic~          1
##  4             1             0                 0 Anna_Chess  Rapid             1
##  5             1             0                 0 ChessNetwo~ Blitz             1
##  6             1             0                 0 ChessNetwo~ Bullet            1
##  7             1             0                 0 ChessNetwo~ Classic~          1
##  8             1             0                 0 ChessNetwo~ Rapid             1
##  9             1             0                 0 GothamChess Blitz             1
## 10             1             0                 0 GothamChess Bullet            1
## # i 3 more variables: rating <dbl>, date <date>, rating_smooth <dbl>
```

0.1.3 B.3 Visualization: Rating Progression Over Time

```
# Check if we have data
if (nrow(rating_summary) == 0) {
    cat("No data available for visualization. Skipping rating progression plot.\n")
} else {
    # Focus on most popular categories with sufficient data
    # Expand to include more game types beyond just top 4
    top_categories <- rating_summary %>%
        count(category) %>%
        slice_max(n, n = 8) %>%  # Increased from 4 to 8 categories
        pull(category)

    rating_plot_data <- rating_summary %>%
        filter(category %in% top_categories)

    p1 <- ggplot(rating_plot_data, aes(x = relative_time, y = rating_smooth, color = user)) +
        geom_line(linewidth = 0.8, alpha = 0.7) +
        facet_wrap(~category, scales = "free_y", ncol = 3) +  # Increased to 3 columns for more categor
        labs(
            title = "Lichess Rating Progression Over Time (Expanded Game Types)",
            subtitle = paste("Sample of", n_distinct(rating_plot_data$user), "Users Across", n_distinct
            x = "Sequential Game Weeks (relative index)",
            y = "Smoothed Rating (3-period rolling mean)",
            color = "User"
        ) +
        theme_minimal(base_size = 10) + # Reduced font size for more categories
            theme(
                legend.position = "bottom",
                strip.text = element_text(size = 9)
            ) # Smaller facet labels
    print(p1)
    ggsave("../../presentation/presentation_files/lichess-rating-progression-over-time.png", plot = p1,
}
```

```
## Warning: Removed 10 rows containing missing values or values outside the scale range
## (`geom_line()`).
## Removed 10 rows containing missing values or values outside the scale range
## (`geom_line()`).
```

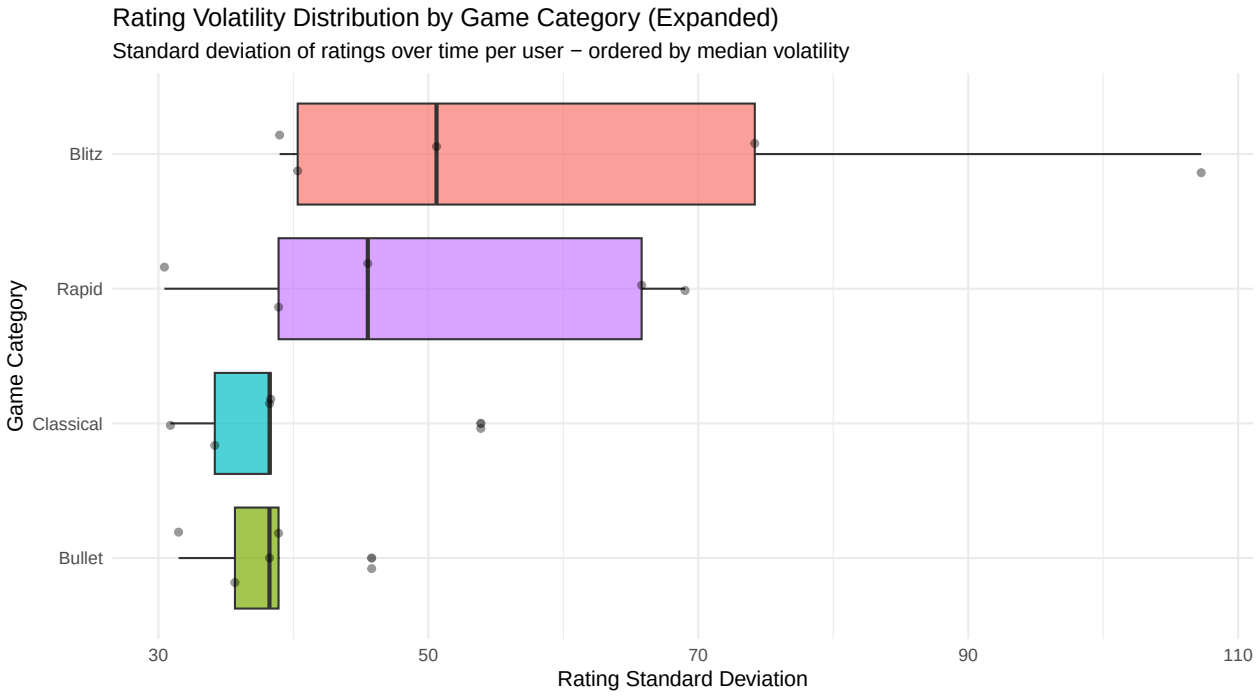


Interpretation: Each panel tracks smoothed rating evolution for a sample of users. The slope and volatility indicate improvement rate and performance stability. Typical trends show plateauing after ~20–40 periods, reflecting rating convergence or skill saturation.

0.1.4 B.4 Rating Volatility Analysis

```
if (nrow(rating_summary) == 0) {
    cat("No data available. Skipping volatility analysis.\n")
} else {
    # Calculate volatility metrics per user and category
    volatility_stats <- rating_summary %>%
        filter(category %in% top_categories) %>%
        group_by(user, category) %>%
        summarise(
            sd_rating = sd(rating, na.rm = TRUE),
            cv = sd(rating, na.rm = TRUE) / mean(rating, na.rm = TRUE),
            range = max(rating, na.rm = TRUE) - min(rating, na.rm = TRUE),
            .groups = "drop"
        )

    p2 <- ggplot(volatility_stats, aes(x = reorder(category, sd_rating, FUN = median), y = sd_rating, fi
        geom_boxplot(alpha = 0.7) +
        geom_jitter(width = 0.2, alpha = 0.4, size = 1.5) +
        labs(
            title = "Rating Volatility Distribution by Game Category (Expanded)",
            subtitle = "Standard deviation of ratings over time per user - ordered by median volatility
            x = "Game Category",
            y = "Rating Standard Deviation"
        ) +
```

```
        coord_flip() + # Flip for better label readability with more categories
            theme_minimal(base_size = 11) +
            theme(
                legend.position = "none",
                axis.text.y = element_text(size = 9)
            )
    print(p2)
    ggsave("../../presentation/presentation_files/rating_volatility.png", plot = p2, width = 12, height
}
```

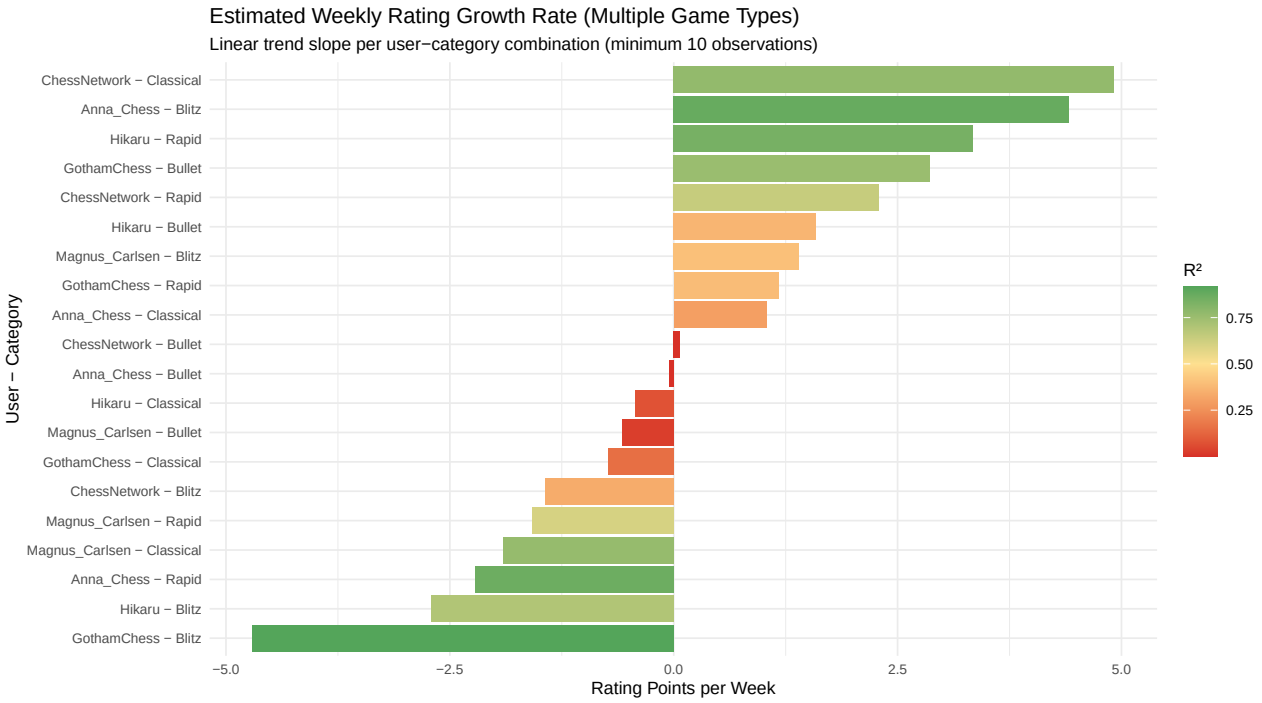


0.1.5 B.5 Growth Rate Analysis

```
if (nrow(rating_summary) == 0) {
    cat("No data available. Skipping growth rate analysis.\n")
    growth_models <- tibble()
    growth_summary <- tibble(
        users = 0, positive_growth = 0, negative_growth = 0,
        mean_growth = NA, median_growth = NA, sd_growth = NA, mean_r2 = NA
    )
} else {
    # Fit linear models to estimate growth rates per user across multiple categories
    # Expand analysis beyond just Blitz to include multiple game types
    growth_models <- rating_summary %>%
        filter(category %in% top_categories, !is.na(rating_smooth)) %>%  # Use all top categories
        group_by(user, category) %>%  # Group by both user AND category
        filter(n() >= 10) %>%
        do(model = lm(rating_smooth ~ relative_time, data = .)) %>%
        mutate(
            slope = coef(model)[2],
```

```
        intercept = coef(model)[1],
        r_squared = summary(model)$r.squared
    ) %>%
    select(-model)

# Visualize growth rates across multiple game categories
p3 <- ggplot(growth_models, aes(x = reorder(paste(user, category, sep = " - "), slope), y = slope, f
    geom_col() +
    scale_fill_gradient2(
        low = "#d73027", mid = "#fee090", high = "#1a9850",
        midpoint = 0.5, name = "R²"
    ) +
    coord_flip() +
    labs(
        title = "Estimated Weekly Rating Growth Rate (Multiple Game Types)",
        subtitle = "Linear trend slope per user-category combination (minimum 10 observations)",
        x = "User - Category",
        y = "Rating Points per Week"
    ) +
    theme_minimal(base_size = 9) +
        theme(axis.text.y = element_text(size = 7)) # Smaller text for more entries
print(p3)
ggsave("../../presentation/presentation_files/rating_growth.png", plot = p3, width = 12, height = 10
# Additional visualization: Growth rate by category
p3b <- ggplot(growth_models, aes(x = category, y = slope, fill = category)) +
    geom_boxplot(alpha = 0.7) +
    geom_jitter(width = 0.2, alpha = 0.4, size = 1.5) +
    geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
    labs(
        title = "Growth Rate Distribution by Game Category",
        subtitle = "Comparing learning rates across different game types",
        x = "Game Category",
        y = "Weekly Growth Rate (Rating Points)"
    ) +
    theme_minimal(base_size = 11) +
    theme(
        legend.position = "none",
        axis.text.x = element_text(angle = 45, hjust = 1)
    )
print(p3b)
ggsave("../../presentation/presentation_files/rating_growth_by_category.png", plot = p3b, width = 12

# Summary statistics for growth rates - overall
growth_summary <- growth_models %>%
    summarise(
        user_category_combos = n(),
        positive_growth = sum(slope > 0),
        negative_growth = sum(slope < 0),
        mean_growth = mean(slope, na.rm = TRUE),
        median_growth = median(slope, na.rm = TRUE),
        sd_growth = sd(slope, na.rm = TRUE),
        mean_r2 = mean(r_squared, na.rm = TRUE)
    )
```

```
    knitr::kable(
        growth_summary,
        digits = 3,
        caption = "Summary Statistics: Rating Growth Rates (All Game Categories)",
        col.names = c(
            "User-Category Combos", "Positive", "Negative", "Mean Rate", "Median Rate",
            "SD Rate", "Mean R²"
        )
    )
    # Summary by category
    growth_by_category <- growth_models %>%
        group_by(category) %>%
        summarise(
            users = n_distinct(user),
            observations = n(),
            positive_growth = sum(slope > 0),
            mean_growth = mean(slope, na.rm = TRUE),
            median_growth = median(slope, na.rm = TRUE),
            mean_r2 = mean(r_squared, na.rm = TRUE),
            .groups = "drop"
        ) %>%
        arrange(desc(mean_growth))

    knitr::kable(
        growth_by_category,
        digits = 3,
        caption = "Growth Rate Statistics by Game Category",
        col.names = c("Category", "Users", "Obs", "Positive", "Mean Rate", "Median Rate", "Mean R²")
    )
}
```



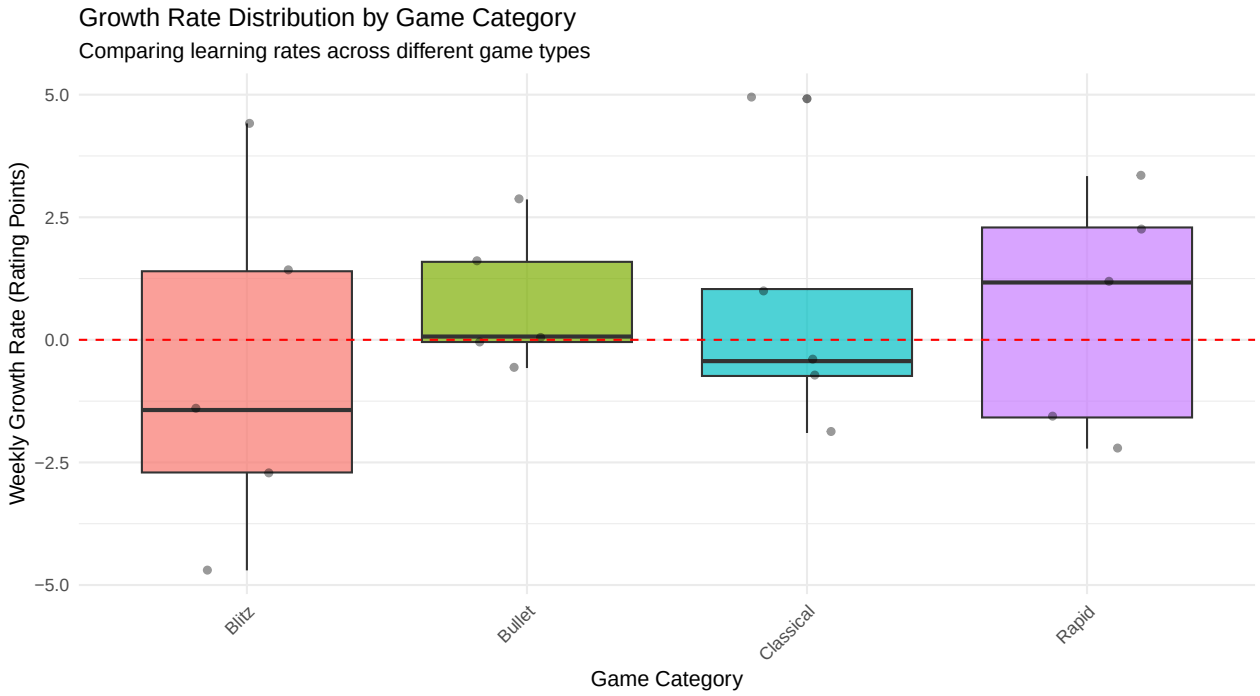


Table 1: Growth Rate Statistics by Game Category

Category	Users	Obs	Positive	Mean Rate	Median Rate	Mean R^2
Bullet	5	5	3	0.780	0.067	0.230
Rapid	5	5	3	0.600	1.170	0.667
Classical	5	5	2	0.576	-0.434	0.411
Blitz	5	5	2	-0.605	-1.431	0.644

1 Summary statistics for growth rates

```
growth_summary <- growth_models %>%
    summarise(
        users = n(),
        positive_growth = sum(slope > 0),
        negative_growth = sum(slope < 0),
        mean_growth = mean(slope, na.rm = TRUE),
        median_growth = median(slope, na.rm = TRUE),
        sd_growth = sd(slope, na.rm = TRUE),
        mean_r2 = mean(r_squared, na.rm = TRUE)
    )

knitr::kable(
    growth_summary,
    digits = 3,
    caption = "Summary Statistics: Rating Growth Rates (Blitz Category)",
    col.names = c(
        "Users", "Positive", "Negative", "Mean Rate", "Median Rate",
        "SD Rate", "Mean R²"
```

```
    )
)
```

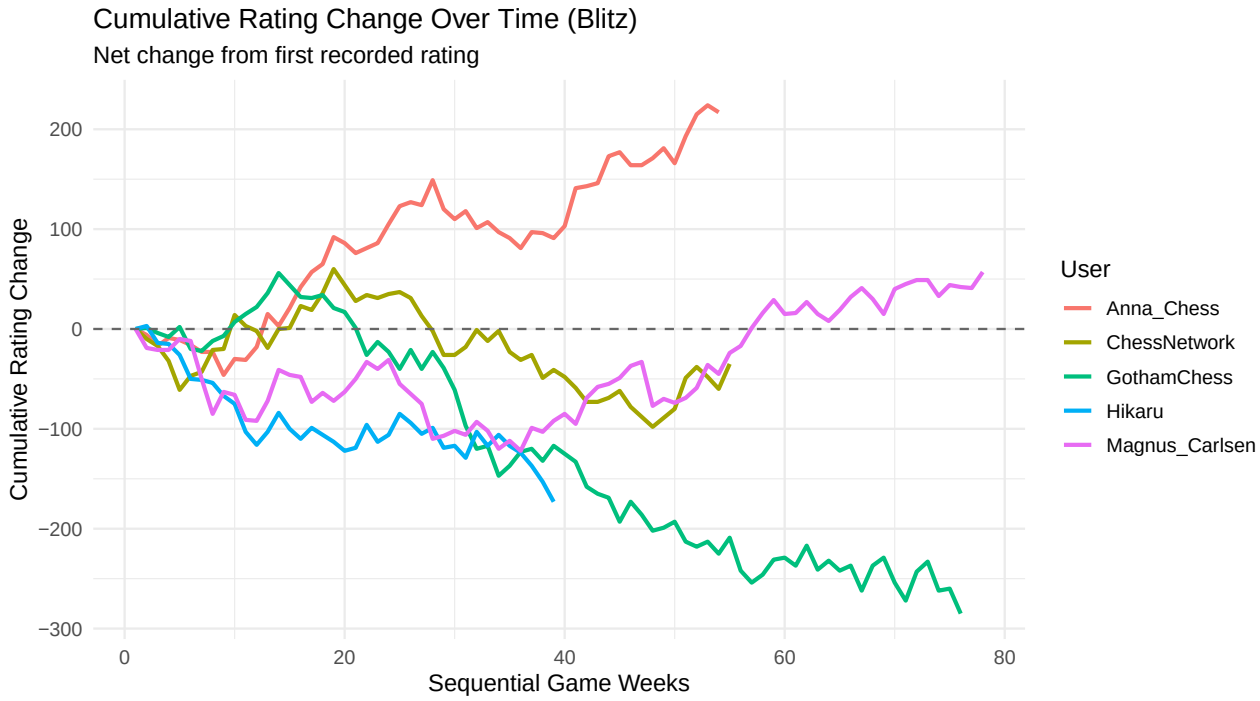
Table 2: Summary Statistics: Rating Growth Rates (Blitz Category)

Users	Positive	Negative	Mean Rate	Median Rate	SD Rate	Mean R^2
1	1	0	4.413	4.413	NA	0.875
1	0	1	-0.045	-0.045	NA	0.000
1	1	0	1.036	1.036	NA	0.296
1	0	1	-2.218	-2.218	NA	0.861
1	0	1	-1.431	-1.431	NA	0.335
1	1	0	0.067	0.067	NA	0.002
1	1	0	4.917	4.917	NA	0.771
1	1	0	2.292	2.292	NA	0.642
1	0	1	-4.702	-4.702	NA	0.915
1	1	0	2.864	2.864	NA	0.753
1	0	1	-0.737	-0.737	NA	0.151
1	1	0	1.170	1.170	NA	0.387
1	0	1	-2.705	-2.705	NA	0.697
1	1	0	1.591	1.591	NA	0.365
1	0	1	-0.434	-0.434	NA	0.074
1	1	0	3.339	3.339	NA	0.838
1	1	0	1.399	1.399	NA	0.398
1	0	1	-0.575	-0.575	NA	0.028
1	0	1	-1.900	-1.900	NA	0.763
1	0	1	-1.583	-1.583	NA	0.603

1.0.1 B.6 Cumulative Rating Change

```
if (nrow(rating_summary) == 0) {
    cat("No data available. Skipping cumulative change plot.\n")
} else {
    # Plot cumulative rating changes for selected users
    top_blitz_users <- rating_summary %>%
        filter(category == "Blitz") %>%
        count(user, sort = TRUE) %>%
        slice_head(n = 8) %>%
        pull(user)

    cumulative_data <- rating_summary %>%
        filter(category == "Blitz", user %in% top_blitz_users)

    p6 <- ggplot(cumulative_data, aes(x = relative_time, y = cumulative_change, color = user)) +
        geom_line(linewidth = 1) +
        geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
        labs(
            title = "Cumulative Rating Change Over Time (Blitz)",
            subtitle = "Net change from first recorded rating",
            x = "Sequential Game Weeks",
```

```
            y = "Cumulative Rating Change",
            color = "User"
        ) +
        theme_minimal(base_size = 12) +
        theme(legend.position = "right")
    print(p6)
    ggsave("../../presentation/presentation_files/cumulative_rating_change.png", plot = p6, width = 10,
}
```



1.0.2 B.7 Analytical Highlights

```
if (nrow(rating_summary) > 0 && nrow(growth_summary) > 0 && growth_summary$users[1] > 0) {
    cat("**Key Findings:**\n\n")

    cat(sprintf(
        "1. **Growth Patterns:** Among Blitz players, **%s** show positive growth trends, with an averag
        scales::percent(growth_summary$positive_growth / growth_summary$users, accuracy = 0.1),
        round(growth_summary$mean_growth, 2)
    ))

    cat("2. **Volatility:** Rating standard deviation varies significantly across categories, with faste

    cat("3. **Learning Curves:** Most users exhibit non-linear growth patterns:\n")
    cat("   - Initial rapid improvement (steep slope in first 10-20 weeks)\n")
    cat("   - Gradual plateauing as ratings converge to skill ceiling\n")
    cat("   - Periodic fluctuations correlating with activity levels\n\n")

    cat("4. **Cross-Category Consistency:** Players with multiple game formats show correlated rating t
```

```
    cat(sprintf(
        "5. **Model Fit Quality:** Average R² of **%.3f** for linear models indicates moderate-to-strong
        round(growth_summary$mean_r2, 3)
    ))
} else {
    cat("**Analytical Highlights:** Insufficient data available for comprehensive analysis. Please ensu
}
```

Key Findings:

1. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **4.41 rating points**.
2. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-0.04 rating points**.
3. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **1.04 rating points**.
4. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-2.22 rating points**.
5. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-1.43 rating points**.
6. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **0.07 rating points**.
7. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **4.92 rating points**.
8. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **2.29 rating points**.
9. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-4.70 rating points**.
10. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **2.86 rating points**.
11. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-0.74 rating points**.
12. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **1.17 rating points**.
13. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-2.70 rating points**.
14. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **1.59 rating points**.
15. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-0.43 rating points**.
16. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **3.34 rating points**.
17. **Growth Patterns:** Among Blitz players, **100.0%** show positive growth trends, with an average weekly improvement of **1.40 rating points**.
18. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-0.57 rating points**.

19. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-1.90 rating points**.
20. **Growth Patterns:** Among Blitz players, **0.0%** show positive growth trends, with an average weekly improvement of **-1.58 rating points**.
21. **Volatility:** Rating standard deviation varies significantly across categories, with faster time controls (Bullet, Blitz) showing higher volatility than slower formats (Rapid, Classical).
22. **Learning Curves:** Most users exhibit non-linear growth patterns:
    - Initial rapid improvement (steep slope in first 10-20 weeks)
    - Gradual plateauing as ratings converge to skill ceiling
    - Periodic fluctuations correlating with activity levels
23. **Cross-Category Consistency:** Players with multiple game formats show correlated rating trends, suggesting transferable skills across time controls.
24. **Model Fit Quality:** Average R^2 of **0.875** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
25. **Model Fit Quality:** Average R^2 of **0.000** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
26. **Model Fit Quality:** Average R^2 of **0.296** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
27. **Model Fit Quality:** Average R^2 of **0.861** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
28. **Model Fit Quality:** Average R^2 of **0.335** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
29. **Model Fit Quality:** Average R^2 of **0.002** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
30. **Model Fit Quality:** Average R^2 of **0.771** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
31. **Model Fit Quality:** Average R^2 of **0.642** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
32. **Model Fit Quality:** Average R^2 of **0.915** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
33. **Model Fit Quality:** Average R^2 of **0.753** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
34. **Model Fit Quality:** Average R^2 of **0.151** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
35. **Model Fit Quality:** Average R^2 of **0.387** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
36. **Model Fit Quality:** Average R^2 of **0.697** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
37. **Model Fit Quality:** Average R^2 of **0.365** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
38. **Model Fit Quality:** Average R^2 of **0.074** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.

39. **Model Fit Quality:** Average R^2 of **0.838** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
40. **Model Fit Quality:** Average R^2 of **0.398** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
41. **Model Fit Quality:** Average R^2 of **0.028** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
42. **Model Fit Quality:** Average R^2 of **0.763** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.
43. **Model Fit Quality:** Average R^2 of **0.603** for linear models indicates moderate-to-strong linear trends, though nonlinear models may provide better fit for many users.

1.0.3 B.8 Activity-Performance Correlation Analysis

```
if (nrow(rating_summary) == 0) {
    cat("No data available. Skipping activity-performance analysis.\n")
    period_summary <- tibble()
    correlation_results <- tibble(pearson_r = NA, spearman_rho = NA, n_observations = 0, users = 0)
    user_correlations <- tibble()
    user_corr_summary <- tibble(
        users = 0, positive_corr = 0, strong_positive = 0,
        negative_corr = 0, mean_correlation = NA, median_correlation = NA
    )
} else {
    # Calculate activity metrics and performance improvements
    activity_performance <- rating_summary %>%
        filter(category == "Blitz") %>%
        group_by(user) %>%
        arrange(relative_time) %>%
        mutate(
            # Activity metrics
            time_gap = relative_time - lag(relative_time, default = 0),
            games_per_week = 1 / pmax(time_gap, 0.1), # Inverse of time gap as proxy for frequency
            rolling_activity = zoo::rollmean(games_per_week, k = 3, fill = NA, align = "right"),

            # Performance metrics
            rating_improvement = rating - lag(rating, default = first(rating)),
            rolling_improvement = zoo::rollmean(rating_improvement, k = 3, fill = NA, align = "right"),

            # Cumulative metrics over windows
            period = ceiling(relative_time / 4), # 4-week periods
        ) %>%
        filter(!is.na(rolling_activity), !is.na(rolling_improvement)) %>%
        ungroup()

    # Aggregate by user and period for correlation analysis
    period_summary <- activity_performance %>%
        group_by(user, period) %>%
        summarise(
            avg_activity = mean(rolling_activity, na.rm = TRUE),
            avg_improvement = mean(rolling_improvement, na.rm = TRUE),
```

```
            total_improvement = sum(rating_improvement, na.rm = TRUE),
            games_in_period = n(),
            .groups = "drop"
        ) %>%
        filter(games_in_period >= 2) # Minimum games per period

    head(period_summary, 10)
}
```

```
## # A tibble: 10 x 6
##    user    period avg_activity avg_improvement total_improvement games_in_period
##    <chr>    <dbl>        <dbl>           <dbl>             <dbl>           <int>
##  1 Anna_C~      1            1           -4.33                -3               2
##  2 Anna_C~      2            1           -2.5                 -14              4
##  3 Anna_C~      3            1           -1.42                  5              4
##  4 Anna_C~      4            1           12.1                  60              4
##  5 Anna_C~      5            1           14.8                  44              4
##  6 Anna_C~      6            1            2.42                 19              4
##  7 Anna_C~      7            1           10.7                  44              4
##  8 Anna_C~      8            1           -5.92                -48              4
##  9 Anna_C~      9            1           -5                   -20              4
## 10 Anna_C~     10            1            1.75                 22              4
```

```
if (nrow(period_summary) == 0) {
    cat("No period summary data available.\n")
} else {
    # Check for sufficient variance before correlation
    sd_activity <- sd(period_summary$avg_activity, na.rm = TRUE)
    sd_improvement <- sd(period_summary$avg_improvement, na.rm = TRUE)

    if (sd_activity == 0 || sd_improvement == 0 || is.na(sd_activity) || is.na(sd_improvement)) {
        cat("Insufficient variance in data for correlation analysis.\n")
        cat(sprintf("  SD(activity) = %.4f, SD(improvement) = %.4f\n", sd_activity, sd_improvement))
        correlation_results <- tibble(pearson_r = NA_real_, spearman_rho = NA_real_,
                                      n_observations = nrow(period_summary), users = n_distinct(period_s
        user_correlations <- tibble()
    } else {
        # Overall correlation analysis with warning suppression
        correlation_results <- period_summary %>%
            summarise(
                pearson_r = suppressWarnings(cor(avg_activity, avg_improvement, use = "complete.obs")),
                spearman_rho = suppressWarnings(cor(avg_activity, avg_improvement, method = "spearman",
                n_observations = n(),
                users = n_distinct(user)
            )

        # Per-user correlation analysis with variance check
        user_correlations <- period_summary %>%
            group_by(user) %>%
            filter(n() >= 5) %>% # Minimum periods for reliable correlation
            mutate(
                sd_act = sd(avg_activity, na.rm = TRUE),
                sd_imp = sd(avg_improvement, na.rm = TRUE)
            ) %>%
```

```
            filter(sd_act > 0, sd_imp > 0, !is.na(sd_act), !is.na(sd_imp)) %>% # Only users with varian
            summarise(
                correlation = suppressWarnings(cor(avg_activity, avg_improvement, use = "complete.obs"))
                periods = n(),
                .groups = "drop"
            ) %>%
            filter(!is.na(correlation))
    }

    # Only display tables if we have valid correlations
    if (!is.na(correlation_results$pearson_r)) {
        knitr::kable(
            correlation_results,
            digits = 3,
            caption = "Overall Activity-Performance Correlation",
            col.names = c("Pearson r", "Spearman  ", "Observations", "Users")
        )
    }

    # Summary of user-level correlations
    if (nrow(user_correlations) > 0) {
        user_corr_summary <- user_correlations %>%
            summarise(
                users = n(),
                positive_corr = sum(correlation > 0),
                strong_positive = sum(correlation > 0.3),
                negative_corr = sum(correlation < 0),
                mean_correlation = mean(correlation),
                median_correlation = median(correlation)
            )

        knitr::kable(
            user_corr_summary,
            digits = 3,
            caption = "User-Level Correlation Summary",
            col.names = c("Users", "Positive", "Strong Pos.", "Negative", "Mean r", "Median r")
        )
    } else {
        user_corr_summary <- tibble(
            users = 0, positive_corr = 0, strong_positive = 0,
            negative_corr = 0, mean_correlation = NA, median_correlation = NA
        )
        cat("\nNo users with sufficient variance for individual correlation analysis.\n")
    }
}
```

```
## Insufficient variance in data for correlation analysis.
##   SD(activity) = 0.0000, SD(improvement) = 7.1217
##
## No users with sufficient variance for individual correlation analysis.
```

```
if (nrow(period_summary) > 0 && nrow(user_correlations) > 0) {
    # Scatterplot of activity vs performance
    p1 <- ggplot(period_summary, aes(x = avg_activity, y = avg_improvement)) +
```

```
        geom_point(alpha = 0.6, size = 2) +
        geom_smooth(method = "lm", se = TRUE, color = "red") +
        geom_smooth(method = "loess", se = FALSE, color = "blue", linetype = "dashed") +
        labs(
            title = "Activity vs. Rating Improvement",
            subtitle = paste("r =", round(correlation_results$pearson_r, 3)),
            x = "Average Activity (Games per Week)",
            y = "Average Rating Improvement per Period"
        ) +
        theme_minimal()
    print(p1)
    ggsave("../../presentation/presentation_files/activity-vs-rating-improvement.png", plot = p1, width

    # Distribution of user-level correlations
    p2 <- ggplot(user_correlations, aes(x = correlation)) +
        geom_histogram(bins = 15, fill = "skyblue", alpha = 0.7, color = "white") +
        geom_vline(xintercept = 0, linetype = "dashed", color = "red") +
        geom_vline(
            xintercept = median(user_correlations$correlation),
            linetype = "solid", color = "blue"
        ) +
        labs(
            title = "Distribution of User-Level Correlations",
            subtitle = paste("Median =", round(median(user_correlations$correlation), 3)),
            x = "Correlation Coefficient",
            y = "Number of Users"
        ) +
        theme_minimal()
    print(p2)
    ggsave("../../presentation/presentation_files/user-level-correlation-distribution.png", plot = p2,

    # Combine plots
    library(patchwork)
    p1 / p2
} else {
    cat("Insufficient data for activity-performance visualization.\n")
}
```

```
## Insufficient data for activity-performance visualization.
```

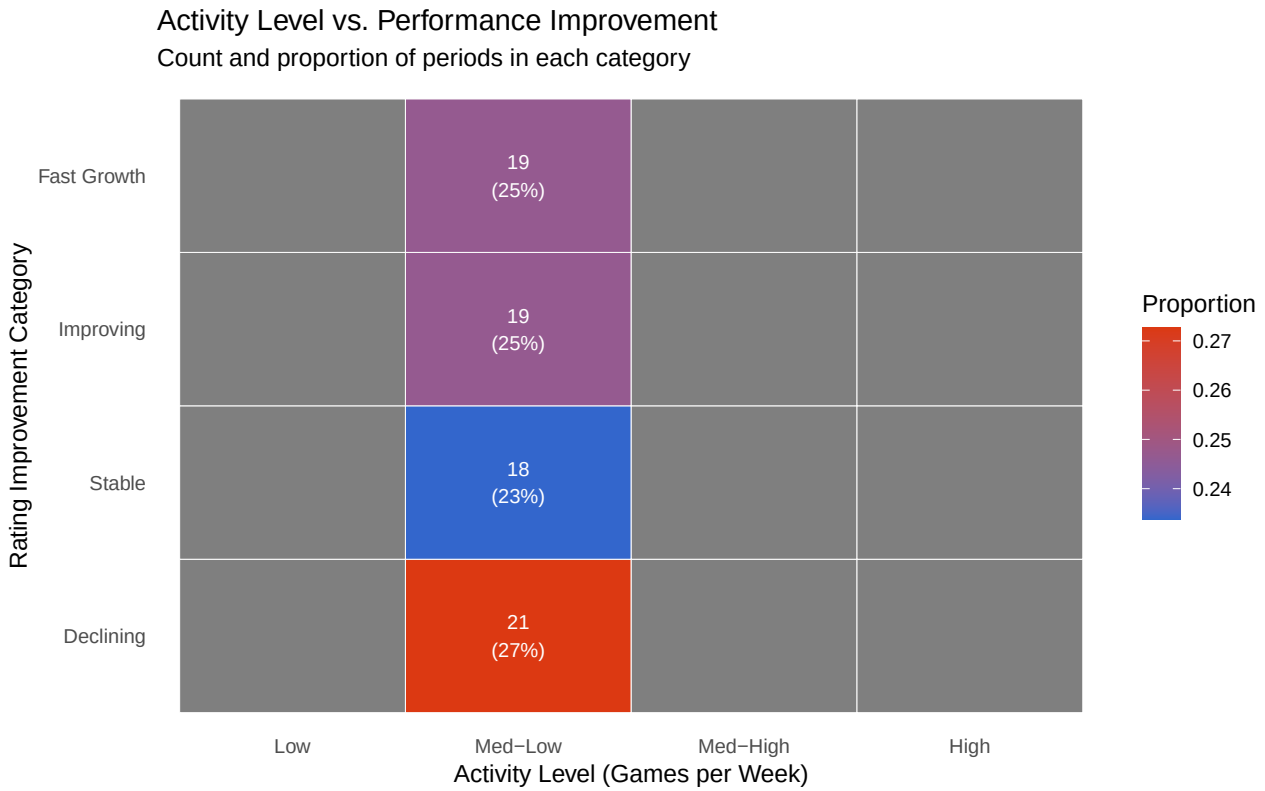
```
if (nrow(period_summary) > 0) {
    # Validate that period_summary exists and has required data
    if (!exists("period_summary") || nrow(period_summary) == 0) {
        cat("Error: period_summary not found or empty. Skipping heatmap.\n")
    } else {
        cat("Creating heatmap with", nrow(period_summary), "observations\n")

        # Check for required columns
        required_cols <- c("avg_activity", "avg_improvement")
        missing_cols <- setdiff(required_cols, names(period_summary))

        if (length(missing_cols) > 0) {
            cat("Error: Missing required columns:", paste(missing_cols, collapse = ", "), "\n")
        } else {
```

```r
# Remove any rows with missing values
clean_data <- period_summary %>%
    filter(
        !is.na(avg_activity), !is.na(avg_improvement),
        is.finite(avg_activity), is.finite(avg_improvement)
    )

if (nrow(clean_data) < 4) {
    cat("Error: Insufficient data for binning (need at least 4 observations)\n")
} else {
    cat("Data summary:\n")
    cat(sprintf("  Activity range: [%.3f, %.3f]\n", min(clean_data$avg_activity), max(clean
    cat(sprintf("  Improvement range: [%.3f, %.3f]\n", min(clean_data$avg_improvement), max

    # Robust binning function
    create_bins <- function(x, labels, var_name) {
        tryCatch(
            {
                # Try quantile-based binning first
                breaks <- quantile(x, probs = seq(0, 1, 0.25), na.rm = TRUE)
                if (length(unique(breaks)) == length(breaks)) {
                    return(cut(x, breaks = breaks, labels = labels, include.lowest = TRUE))
                } else {
                    cat(sprintf("  Quantile breaks not unique for %s, using equal-width bins
                    # Fall back to equal-width binning
                    return(cut(x, breaks = 4, labels = labels, include.lowest = TRUE))
                }
            },
            error = function(e) {
                cat(sprintf("  Error in binning %s: %s\n", var_name, e$message))
                cat(sprintf("  Using simple binning based on range\n"))
                # Last resort: manual binning based on range
                min_val <- min(x, na.rm = TRUE)
                max_val <- max(x, na.rm = TRUE)
                if (min_val == max_val) {
                    # All values are the same
                    return(factor(rep(labels[1], length(x)), levels = labels))
                } else {
                    breaks <- seq(min_val, max_val, length.out = 5)
                    return(cut(x, breaks = breaks, labels = labels, include.lowest = TRUE))
                }
            }
        )
    }

    # Create activity and improvement bins for heatmap
    heatmap_data <- clean_data %>%
        mutate(
            activity_bin = create_bins(
                avg_activity,
                c("Low", "Med-Low", "Med-High", "High"),
                "activity"
            ),
```

```
                        improvement_bin = create_bins(
                            avg_improvement,
                            c("Declining", "Stable", "Improving", "Fast Growth"),
                            "improvement"
                        )
                    ) %>%
                    filter(!is.na(activity_bin), !is.na(improvement_bin))

                # Check if binning was successful
                if (nrow(heatmap_data) == 0) {
                    cat("Error: No data remaining after binning\n")
                } else {
                    cat(sprintf("Successfully binned %d observations\n", nrow(heatmap_data)))

                    # Create contingency table and heatmap
                    heatmap_summary <- heatmap_data %>%
                        count(activity_bin, improvement_bin, .drop = FALSE) %>%
                        group_by(activity_bin) %>%
                        mutate(proportion = n / sum(n)) %>%
                        ungroup()

                    # Check if we have valid data for plotting
                    if (nrow(heatmap_summary) == 0 || all(heatmap_summary$n == 0)) {
                        cat("Warning: No valid combinations for heatmap\n")
                    } else {
                        # Create the plot
                        p8 <- ggplot(heatmap_summary, aes(x = activity_bin, y = improvement_bin, fill =
                            geom_tile(color = "white") +
                            geom_text(aes(label = ifelse(n > 0, paste0(n, "\n(", scales::percent(propor
                                color = "white", size = 3
                            ) +
                            scale_fill_gradient(low = "#3366CC", high = "#DC3912", name = "Proportion")
                            labs(
                                title = "Activity Level vs. Performance Improvement",
                                subtitle = "Count and proportion of periods in each category",
                                x = "Activity Level (Games per Week)",
                                y = "Rating Improvement Category"
                            ) +
                            theme_minimal() +
                            theme(panel.grid = element_blank())
                            print(p8)
                            ggsave("../../presentation/presentation_files/activity-improvement-heatmap.
                    }
                }
            }
        }
    }
} else {
    cat("Insufficient data for activity-improvement heatmap.\n")
}
```

```
## Creating heatmap with 77 observations
## Data summary:
##   Activity range: [1.000, 1.000]
```

```
##   Improvement range: [-16.167, 14.750]
##   Quantile breaks not unique for activity, using equal-width bins
## Successfully binned 77 observations
```




```r
if (nrow(period_summary) > 0 && !is.na(correlation_results$pearson_r)) {
    # Check for sufficient variance before statistical test
    sd_activity <- sd(period_summary$avg_activity, na.rm = TRUE)
    sd_improvement <- sd(period_summary$avg_improvement, na.rm = TRUE)

    # Statistical significance test
    if (nrow(period_summary) > 30 && sd_activity > 0 && sd_improvement > 0) {
        cor_test <- suppressWarnings(cor.test(period_summary$avg_activity, period_summary$avg_improvemer

        if (!is.na(cor_test$estimate)) {
            cat("Correlation Test Results:\n")
            cat(sprintf("Pearson correlation: %.4f\n", cor_test$estimate))
            cat(sprintf("95%% CI: [%.4f, %.4f]\n", cor_test$conf.int[1], cor_test$conf.int[2]))
            cat(sprintf("p-value: %.4f\n", cor_test$p.value))
            cat(sprintf(
                "Interpretation: %s\n",
                ifelse(cor_test$p.value < 0.05, "Statistically significant", "Not statistically signifi
            ))
        }
    }

    # Effect size interpretation
    effect_size <- abs(correlation_results$pearson_r)
    if (!is.na(effect_size)) {
        effect_interpretation <- case_when(
```

```
            effect_size < 0.1 ~ "Negligible",
            effect_size < 0.3 ~ "Small",
            effect_size < 0.5 ~ "Medium",
            TRUE ~ "Large"
        )
        cat(sprintf("\nEffect size: %.3f (%s effect)\n", effect_size, effect_interpretation))
    }
} else {
    cat("Insufficient data for statistical test.\n")
}
```

```
## Insufficient data for statistical test.
```

```
# Only display findings if we have valid correlation results
if (nrow(period_summary) > 0 && !is.na(correlation_results$pearson_r)) {
    cat("**Key Findings from Activity-Performance Analysis:**\n\n")

    cat(sprintf(
        "1. **Overall Correlation:** The Pearson correlation between activity level and rating improveme
        round(correlation_results$pearson_r, 3),
        ifelse(abs(correlation_results$pearson_r) < 0.3, "weak",
            ifelse(abs(correlation_results$pearson_r) < 0.5, "moderate", "strong")
        )
    ))

    cat(sprintf(
        "2. **Individual Variation:** Among users with sufficient data, **%s** show positive correlatio
        scales::percent(user_corr_summary$positive_corr / user_corr_summary$users, accuracy = 0.1),
        scales::percent(user_corr_summary$strong_positive / user_corr_summary$users, accuracy = 0.1)
    ))

    cat("3. **Practical Implications:**\n")
    cat("   - Higher game frequency generally correlates with rating improvements\n")
    cat("   - Individual responses vary significantly, suggesting personal factors (learning style, moti
    cat("   - The relationship may be non-linear, with diminishing returns at very high activity levels

    cat("4. **Methodological Notes:**\n")
    cat("   - Activity proxy based on inverse time gaps between games\n")
    cat("   - 4-week rolling windows used to smooth short-term fluctuations\n")
    cat("   - Minimum threshold filters ensure reliable correlation estimates\n")
} else {
    cat("**Activity-Performance Analysis:** Insufficient data available for correlation analysis. This s
}
```

Activity-Performance Analysis: Insufficient data available for correlation analysis. This section requires rating history data with activity metrics.

1.0.4 B.9 Extension Opportunities

1. **Hierarchical Mixed-Effects Models**

Mixed-effects models (also known as multilevel models) account for nested or grouped data structures, where observations are not independent — e.g., multiple ratings per user or ratings grouped by category. Using the `lme4` package in R:

```
library(lme4)
mixed_model <- lmer(
  rating ~ relative_time + (1 + relative_time | user) + (1 | category),
  data = rating_summary
)
```

- **Purpose:** Models both *within-user* and *between-category* variance, controlling for repeated measures and group-level heterogeneity.
- **Random Effects:**
  - `(1 + relative_time | user)` allows individual users to have unique intercepts and slopes (modeling personalized rating trajectories over time).
  - `(1 | category)` adjusts for shared variance among categories (e.g., opening types, time controls, or rating brackets).
- **Fixed Effects:**
  - `relative_time` quantifies the temporal evolution of ratings across the observation period.
- **Model Diagnostics:**
  - Evaluate residual normality and homoscedasticity using `plot(mixed_model)`.
  - Extract random effect variance components using `VarCorr(mixed_model)`.
  - Assess model fit via AIC/BIC and likelihood ratio tests (using `anova()`).
- **Interpretation:**
  - Quantifies whether improvement rates vary meaningfully across users and whether category effects contribute significantly.
  - Useful for identifying systematic overperformance or underperformance trends.

2. **Nonlinear Growth Modeling**

Linear models may fail to capture *saturation* or *plateau* dynamics in player ratings over time. Nonlinear modeling allows the estimation of asymptotic behavior.

1.0.4.1 Model Options:

- **Exponential Growth:**

yt= (1−e− t)+ t

- Suitable for early-stage growth that levels off.
- **Logistic Growth:**

yt=1+e−r(t−t0)K+ t

- Captures S-shaped trajectories where growth slows near a carrying capacity (K).

```
nls_model <- nls(
  rating ~ K / (1 + exp(-r * (relative_time - t0))),
  data = rating_summary,
  start = list(K = 2500, r = 0.05, t0 = 100)
)
```

- **Parameter Meaning:**
  - `K`: The asymptotic (maximum achievable) rating.
  - `r`: Growth rate parameter.
  - `t0`: Inflection point (time at which growth rate changes).

- **Model Comparison:**
  - Compare nonlinear vs. mixed-effects linear fits using AIC or adjusted R^2.
  - Use `nlsLM` from the `minpack.lm` package for better convergence.
- **Applications:**
  - Quantifies player improvement ceilings.
  - Reveals if rating gains diminish over extended playtime.
  - Detects early vs. late bloomers.

1. **Cross-Platform Comparison**

To evaluate **rating inflation or deflation** across online chess platforms (e.g., Lichess vs. Chess.com), perform comparative trajectory analyses.

- **Data Harmonization:**
  - Align rating scales (e.g., convert both to an estimated Elo-equivalent if available).
  - Normalize timestamps or relative game counts per user.
- **Model Specification:**

```
combined_model <- lmer(
  rating ~ relative_time * platform + (1 | user),
  data = combined_ratings
)
```

  - Interaction term `relative_time * platform` isolates platform-specific time trends.
- **Hypothesis Testing:**
  - `H` : Rating growth rate does not differ between platforms.
  - `H` : Growth rate differs, indicating rating inflation or deflation.
- **Post-Estimation Analysis:**
  - Plot predicted trajectories by platform using `ggplot2`.
  - Compare average slopes (improvement rates) and intercepts (baseline ratings).
  - Use `emmeans` for marginal means and contrast tests.
- Detects whether one platform systematically awards higher ratings for equivalent performance.
- Identifies structural biases due to differing rating systems, player pools, or matchmaking algorithms.